Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Proyecto del curso · Catedrático: René Ornélyz

ER-AFD

Convertidor de expresiones regulares a autómatas finitos deterministas

Manual de usuario

Integrantes

Carlos Abraham Méndez Asencio · Carné 9490-24-17370

Maycol Alexander Ortiz Flores · Carné 9490-24-27213

Guatemala, noviembre de 2026

1. Requisitos

- Linux de 64 bits (cualquier distribución).
- Python 3.10 o superior (`python3 --version`).
- GNU Make y Git. En Ubuntu: `sudo apt install make git`, si no están instalados.

El programa solo usa la biblioteca estándar de Python, no instala paquetes y no necesita permisos de root.

2. Instalación

```
git clone https://github.com/Medec92227/er-afd.git
cd er-afd
make config
make build
```

`make build` crea dos ejecutables en `dist/`: `er-afd` y `er-lex`. Para llamarlos por su nombre desde cualquier carpeta, se agrega `dist/` al `PATH` de la terminal:

```
export PATH="$PWD/dist:$PATH"
```

Sin ese paso también funcionan como `./dist/er-afd` o `python3 -m er_afd`.

Comando	Qué hace
`make config`	Revisa que la versión de Python sea compatible.
`make build`	Genera `dist/er-afd` y `dist/er-lex`.
`make run`	Ejecuta el ejemplo `(a|b)*abb` con `aabb`. Se cambia con `make run ER='a*b' CADENA='aab'`.
`make test`	Corre las pruebas unitarias.
`make metricas`	Calcula las métricas de precisión.
`make clean`	Borra `build/`, `dist/` y las cachés de Python.

3. Uso

```
er-afd <ER> <cadena>
```

Conviene escribir la ER entre comillas simples, para que la terminal no interprete `*`, `|`, `(` ni `\`. Si la ER o la cadena tienen espacios se usa `\` o comillas dobles:

```
er-afd '(a|b)*abb' 'aabb'
er-afd hola\ mundo hola\ mundo
er-afd "hola mundo" "hola mundo"
```

Qué muestra

1. El AFND: `Q` (estados q0, q1, …), `Σ`, `F` (estados finales) y una línea `qi|símbolo|{destinos}` por transición.
2. El AFD: `Q` (estados Q0, Q1, …), `Σ`, `F` y una línea `Qi|símbolo|Qj` por transición.
3. El recorrido de la cadena, una línea `Qi|carácter|Qj` por cada carácter leído.
4. `Cadena aceptada` o `Cadena rechazada`.

`q0` y `Q0` son siempre los estados iniciales. Si en el AFD un carácter no tiene transición desde el estado actual, el recorrido se detiene en ese punto y la cadena se rechaza.

```
$ er-afd 'ab*' 'abb'
...
Q0|a|Q1
Q1|b|Q2
Q2|b|Q2
Cadena aceptada
$ echo $?
1
```

Código de retorno

Código	Significado
1	La cadena fue aceptada.
0	La cadena fue rechazada.
2	Error: ER mal escrita, carácter fuera de ASCII o archivo que no existe.

Opciones

Opción	Uso
`--verbose`	Muestra también qué estados del AFND forman cada estado del AFD.
`--file RUTA`	Lee la cadena desde un archivo, en lugar del segundo argumento.
`--json`	Imprime los autómatas y el recorrido en JSON.
`--dot PREFIJO`	Crea `PREFIJO.afnd.dot` y `PREFIJO.afd.dot` para dibujarlos con Graphviz (`dot -Tpng`).
`--max-states N`	Máximo de estados del AFD (4096 por defecto).

4. Sintaxis de las expresiones

Símbolo	Significado	Ejemplo
`ab`	Concatenación	`hola`
`a|b`	Unión	`si|no`
`a*`	Cero o más repeticiones	`(ab)*`
`(r)`	Agrupación	`(a|b)c`
`.`	Cualquier carácter	`a.b`
`ε`	Cadena vacía	`(a|ε)b`
`[abc]`	Uno de los caracteres	`[aeiou]`
`[a-z]`	Rango	`[0-9A-F]`
`[^abc]`	Cualquier carácter excepto esos	`[^0-9]`
`\c`	El carácter `c` literal	`\*`, `\\|`, `\\`
`\N`	Un dígito	`\N\N\N\N`
`\A` / `\a` / `\z`	Mayúscula / minúscula / letra	`\A\a*`
`\hXX`	Carácter por su código hexadecimal	`\h20` (espacio)

No existen `+`, `?` ni `{m,n}`: se escriben como texto literal. Para "uno o más" se usa `rr*`; por ejemplo, `\N\N*` son uno o más dígitos.

5. Tokenizar un archivo (er-lex)

Las reglas se escriben en un JSON, en orden de prioridad:

```
[
  {"name": "ESPACIO", "regex": "[\\h20\\h09\\h0A\\h0D][\\h20\\h09\\h0A\\h0D]*", "skip":
true},
  {"name": "SI", "regex": "if"},
  {"name": "IDENTIFICADOR", "regex": "(\\z|_)(\\z|\\N|_)*"},
  {"name": "NUMERO", "regex": "\\N\\N*"}
]
```

```
er-lex --rules examples/reglas.json --file examples/programa.txt
er-lex --rules examples/reglas.json --text 'if x = 10;' --json
```

Devuelve 0 si pudo tokenizar todo el texto y 2 si encuentra un carácter que ninguna regla reconoce.

6. Problemas comunes

Problema	Solución
`make: command not found`	Instalar make: `sudo apt install make`.
`er-afd: command not found`	Ejecutar `make build` y `export PATH="$PWD/dist:$PATH"`, o usar `./dist/er-afd`.
La terminal cambia la ER	Escribir la ER entre comillas simples: `'(a|b)*'`.
`Error: … no ASCII`	La cadena tiene tildes o ñ. El programa trabaja con ASCII (códigos 0 a 127).
Un script con `set -e` se detiene al aceptar	Es normal: la aceptación devuelve 1. Revisar `$?` en lugar de usar `set -e`.